
◆ TRAIN MODEL

```
model = MultinomialNB(alpha=0.1)
model.fit(X_train, y_train)
```

◆ LINE 1

```
model = MultinomialNB(alpha=0.1)
```

What is `MultinomialNB`?

```
MultinomialNB
```

= Machine Learning algorithm (Naive Bayes)

👉 Used for:

- text classification
- spam detection
- sentiment analysis

What is `NB`?

```
NB = Naive Bayes
```

It is a probability-based algorithm.

It learns:

```
P(SPAM | words)
P(HAM | words)
```

What is `alpha=0.1`? 📕

```
alpha
```

= smoothing parameter

👉 prevents zero probability problem

Example:

If a word never appears in training:

```
without alpha → probability becomes 0 ❌
with alpha → small value is added ✔
```

So model doesn't break.

LINE 2

```
model.fit(X_train, y_train)
```

This is where actual learning happens.

WHAT DOES fit() DO?

```
fit = training step
```

It tells model:

```
Here are features (X_train)
Here are correct answers (y_train)
Now learn pattern between them
```

WHAT ARE X_train AND y_train?

```
X_train → email features (TF-IDF numbers)
y_train → labels (0 = HAM, 1 = SPAM)
```

Example:

```
X_train:
[0.2, 0.8, 0.1] → SPAM
[0.9, 0.1, 0.4] → HAM
```

WHAT HAPPENS INSIDE MODEL?

Model calculates:

```
Which words appear more in SPAM emails
Which words appear more in HAM emails
```

It builds probability tables like:

```
money → high chance of SPAM
free → high chance of SPAM
meeting → high chance of HAM
```

FINAL RESULT OF TRAINING

After:

```
model.fit()
```

Model is now trained and can predict:

```
new email → SPAM or HAM
```

COMPLETE FLOW

```
X_train + y_train
↓
MultinomialNB learns patterns
↓
Build probability model
↓
Ready for prediction
```

SUMMARY

```
MultinomialNB → ML algorithm used for text classification

alpha=0.1 → smoothing to avoid zero probability

fit(X_train, y_train) → trains model using features + labels

👉 Use: learn relationship between email words and spam/ham labels
```
